

Final Project Report

Report Details

Submission Date: 05/April/2026

Course: Object Oriented Programming (OOP) Java

Taught by: Mr. Lay Vathna & Mr. Natt KORAT

Project member

Yan Sovanpisoth IDTB110266

Siv Kimleng IDTB110072

Kong Sothearith IDTB110185

Kov Cheaching IDTB110219

Arrangement of Contents

1. Problem Statement
2. Introduction
3. Literature Review
4. Implementation
5. Results
6. Conclusion
7. Appendix

1. Problem Statement

Many individuals struggle to manage their personal finances effectively, especially in tracking income and daily expenses. Traditional methods like manual recording are often inefficient and error-prone. Without a proper system, users may lose control of their spending and fail to maintain accurate financial records.

Therefore, a simple and structured application is needed to help users track transactions, manage balances, and improve financial awareness.

2. Introduction

Background of the Project

The Personal Budget Tracker project was developed to address this issue by providing a structured and user-friendly system that allows users to manage their financial activities digitally. The application focuses on tracking income, expenses, and account balances in an organized way.

Objective

The main objective of this project is to develop a Personal Budget Tracker using Java and object-oriented programming principles. The system is intended to help users:

- record income and expenses
- manage account balance and savings
- set a spending limit
- create savings goals or wishlist items
- improve financial awareness through organized records

Another objective is to apply OOP concepts such as Encapsulation, Inheritance, Abstraction, and Polymorphism in a practical software project.

The Project Impact

This project can benefit users by helping them monitor and control their personal finances more effectively. Instead of recording transactions manually, users can use the system to maintain a more accurate and structured financial record. The project also has academic value because it demonstrates how object-oriented programming, graphical user interfaces, and database systems can be integrated into one complete application. For students, it serves as a practical example of how programming skills can be used to solve real-life problems.

3. Literature Review

Similar Projects

Several existing applications and systems have been developed to help users manage their finances. These projects share common goals such as tracking expenses, budgeting, and improving financial awareness:

- YNAB (You Need A Budget)
- Goodbudget
- Banking Applications

Inspiration and Extension

These applications inspired the Personal Budget Tracker because they show the importance of structured budgeting and expense monitoring. However, this project is designed as a simpler academic system for learning and implementation purposes. It brings together key features such as:

- user signup and login
- passkey-protected transactions
- income and expense tracking
- savings management
- budget limit setting
- wishlist or savings goal management

4. Implementation

Task Breakdowns and Responsible Person

The project was divided among team members so that each person contributed to different parts of the system. Here is the task distribution among our team members:

- Yan Sovanpisoht: Designed the overall project structure, developed the core Java classes, applied object-oriented programming concepts, and contributed to writing the project report.
- Siv Kimleng: Developed the main graphical user interface, including the dashboard, applied object-oriented programming concepts, and contributed to writing the project report.

- Kong Sothearith: Designed the database structure, implemented SQL integration, contributed to the application of object-oriented programming concepts, and prepared the slide presentation.
- Kov Cheaching: Developed the graphical user interface for the login and signup features, contributed to the application of object-oriented programming concepts, and contributed to preparing the slide presentation.

System Design

The Personal Budget Tracker was developed using **Java** as the main programming language. **Maven** was used for project management and dependency handling. **MySQL** was used as the database for storing user accounts, balances, transaction records, and wishlist data. **Java Swing** was used to build the graphical user interface.

The system follows a modular design:

- models package stores the data structures such as User, Account, Transaction Records, and Wishlist items
- service package contains the business logic for handling income, expenses, savings, limits, and wishlist operations
- repository package manages communication with the database
- config package provides the database connection setup
- GUI package contains the user interface screens such as login, signup, and dashboard

This design makes the system easier to maintain, understand, and extend.

Project Structure and Components

The project contains several main components:

1. The first component is **User Management**, which handles signup and login. Users must create an account with a username, email, password, and 4-digit passkey.
2. The second component is **Account Management**, which stores the user's current balance, savings balance, and spending limit.
3. The third component is **Transaction Management**, which records income and expenses. Expense transactions also include categories.
4. The fourth component is **Savings Management**, which allows users to move money into savings and use savings when necessary.

5. The fifth component is **Wishlist Management**, which allows users to create savings goals for specific items.
6. The sixth component is the **Graphical Dashboard**, which displays transaction records in a table and shows summaries such as total income, total expense, total budget, saving balance, and limit balance.

5. Results

The project successfully implemented most of the planned features. The completed features include:

- user signup/login
- income recording
- expense recording with category
- savings deposit
- savings withdrawal
- spending limit setting
- wishlist or savings goal creation
- transaction summary display in the GUI

The core objectives of the project were achieved. The application is functional and demonstrates a complete budgeting workflow from account creation to financial tracking.

Challenges Faced

During development, the team faced several challenges. One challenge was integrating Java Swing with the database for the GUI session. Another challenge was ensuring that logic, such as balances, savings, and limits, was updated correctly after each transaction.

Some features could still be improved in the future. For example, stronger password security, automatic report generation, charts, and deeper database synchronization would make the system more complete.

From these challenges, the team learned the importance of planning, testing, teamwork, and code organization.

6. Conclusion

Perspective

This project provided valuable experience in applying object-oriented programming concepts to a real-world problem. Through this project, the team gained a better understanding of class design, inheritance, encapsulation, modular programming, and database integration.

Challenges

The main challenges included designing a clear system structure, connecting the GUI with the database, and ensuring that financial calculations were handled correctly. These challenges were addressed through teamwork, discussion, repeated testing, and step-by-step improvement of the code.

Final Thoughts

Overall, the Personal Budget Tracker is a useful project that helps users manage income, expenses, savings, and financial goals in a more organized way. Although it is a student project, it demonstrates the practical value of object-oriented programming and shows how software can be used to solve everyday problems. The project is significant both as a functional budgeting system and as a learning experience in Java OOP development.

7. Appendix

Github: <https://github.com/Sotchi10/Personal-Budget-Tracker.git>

Flow Charts:

